Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

"PRODUCT SEARCH AND RECOMMENDATION SYSTEM FOR SHOPPING"

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

TEAM 4

CHERUKUMUDI SRI DHRUTHI : 2520030490

KACHANA MAHIMA : 2520030593

DEEPIKAA : 2520030137

Under the Guidance of

Dr. V. Sireesha

Professor

Department of Computer Science and Engineering

2026-27

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

DECLARATION

The Project Report entitled "Product Search and Recommendation System for Shopping" is a record of bonafide work of CHERUKUMUDI SRI DHRUTHI – 2520030490, KACHANA MAHIMA – 2520030593, and DEEPIKAA – 2520030137, submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The work presented in this report describes the design, implementation, testing, and documentation of a Java-based product search and recommendation application using Data Structures and Algorithms.

The results embodied in this report are based on the submitted project implementation and the project data and test evidence available with the team.

CHERUKUMUDI SRI DHRUTHI – 2520030490

KACHANA MAHIMA – 2520030593

DEEPIKAA – 2520030137

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the mini project based report entitled "Product Search and Recommendation System for Shopping" is a bonafide work done and submitted by CHERUKUMUDI SRI DHRUTHI – 2520030490, KACHANA MAHIMA – 2520030593, and DEEPIKAA – 2520030137 in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering, during the academic year 2026-27.

Signature of the Guide **Signature of the HOD**

(Dr. V. Sireesha)

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

ACKNOWLEDGEMENT

We express our sincere gratitude to our guide, Dr. V. Sireesha, Professor, Department of Computer Science and Engineering, for her valuable guidance, suggestions, and continuous support throughout the development of the Product Search and Recommendation System for Shopping.

We also express our sincere thanks to Dr. P. Venkateswara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the necessary academic support and facilities for completing this project.

We are grateful to the Department of Computer Science and Engineering for providing an environment that enabled us to apply Data Structures and Algorithms to a practical product-search and recommendation problem.

Finally, we thank all the team members for their cooperation and contribution in algorithm implementation, dataset preparation, Java Swing interface development, testing, debugging, documentation, and presentation.

CHERUKUMUDI SRI DHRUTHI – 2520030490

KACHANA MAHIMA – 2520030593

DEEPIKAA – 2520030137

Abstract

The Product Search and Recommendation System for Shopping is a Java-based application developed to make online shopping easier, faster, and more efficient. It helps users discover suitable products based on their requirements, preferences, and budget through an interactive graphical user interface developed using Java Swing.

The application provides several features, including category-based product browsing, requirement-based search, price filtering, fuzzy searching, product comparison, and similarity-based recommendations. Users can explore product details, identify products that match their needs, and receive budget-aware recommendations. The system also provides aspect-based matching, randomized product suggestions, and similar-product discovery to improve the shopping experience.

A key feature of the application is its explainable recommendation mechanism, which helps users understand why a particular product is recommended. It evaluates matching keywords, relevant features, description similarity, category compatibility, and budget suitability. Additional functionalities include wishlist management, shopping cart operations, checkout, order history, and reordering, allowing users to manage their shopping activities conveniently.

The system incorporates various Data Structures and Algorithms techniques, including string matching, dynamic programming, graph-based matching, optimization, randomized algorithms, and parallel processing. Product information and user-related data are managed through file handling, without requiring a database or external libraries.

Overall, the project demonstrates the practical application of Data Structures and Algorithms in developing an interactive shopping system that improves product discovery, simplifies product comparison, provides meaningful recommendations, and supports informed purchasing decisions.

Table Of Contents

CONTENTS	PAGE
CHAPTER 1 - INTRODUCTION	**8**
1.1 Overview	8
1.2 Problem Statement	9
1.3 Scope of the Project	9
1.4 Objectives	10
CHAPTER 2 - SYSTEM REQUIREMENTS	**11**
2.1 Functional Requirements	11
2.2 Non-Functional Requirements	12
2.3 Hardware and Software Requirements	13
2.4 Dataset Requirements	14
CHAPTER 3 - SYSTEM DESIGN	**15**
3.1 High-Level Architecture	15
3.2 System Flowchart	16
3.3 Data and Component Description	16
3.4 DSA and CO Mapping	17
3.5 Dataset Design	18
CHAPTER 4 - IMPLEMENTATION	**19**
4.1 Module-Wise Implementation	19
4.2 PSEUDOCODE	20
4.3 JAVA SWING INTERFACE	24
CHAPTER 5 - RESULTS	**25**
5.1 Testing Strategy	25
5.2 Functional Results	26
5.2.1 Product Catalogue Loading	26
5.2.2 Product Browsing and Category Selection	26
5.2.3 Requirement-Based Product Search	27
5.2.5 Budget-Based Recommendation	28
5.2.6 Aspect-Based Product Matching	29
5.2.7 Randomized Product Recommendation	29
5.2.8 Similar Product Recommendation	30
5.2.9 Shopping Cart and Checkout	30
CHAPTER 6 — CHALLENGES & LIMITATIONS	**34**
6.1 Issues Faced During Development	34
6.2 Solutions Applied	34
6.3 Limitations	35
CHAPTER 7 — CONCLUSION & FUTURE WORK	**36**
7.2 Future Work	36
7.3 Skills Learned	37
REFERENCES	**38**

List of Figures

Figure No.	Caption
Fig. 1.1	Overview of the Product Search and Recommendation System
Fig. 3.1	System Architecture of the Product Search and Recommendation System
Fig. 3.2	System Flowchart
Fig. 3.3	DSA and CO Mapping
Fig. 3.4	Product Dataset Record Structure
Fig. 4.1	Java Swing Shopping Dashboard
Fig. 5.1	Product Browsing and Category Selection
Fig. 5.2	Requirement-Based Product Search
Fig. 5.3	Product Details and Explainable Recommendation
Fig. 5.4	Budget-Based Product Recommendation using Knapsack
Fig. 5.5	Aspect-Based Product Matching
Fig. 5.6	Randomized Product Recommendation
Fig. 5.7	Similar Product Recommendation
Fig. 5.8	Shopping Cart and Checkout
Fig. 5.9	Product Comparison of Selected Products
Fig. 5.10	Wishlist Management
Fig. 5.11	Checkout Confirmation
Fig. 5.12	Order History and Reordering

CHAPTER 1 - INTRODUCTION

1.1 Overview

Product Search and Recommendation System for Shopping is a Data Structures and Algorithms project that applies different algorithmic techniques to searching, retrieving, comparing, matching, and recommending products from a large shopping catalogue. The project combines string matching, fuzzy matching, similarity calculation, graph-based matching, optimization, randomization, and parallel processing within a single application.

The application works with a CSV-based product corpus stored in data/products.csv. The current dataset contains 5,000 product records. Each record is loaded into a Product object and made available to the repository, search engine, recommendation modules, and graphical interface. The repository also builds a keyword index so that search operations can work with prepared product information rather than repeatedly reparsing the file.

The user-facing application is provided through a Java Swing graphical interface. The GUI presents a shopping-oriented workflow with Login/Register, category selection, Browse, Search, Recommendations, Cart, and Checkout. The internal DSA implementations remain behind the interface so that users interact with product-search functions instead of directly selecting algorithms.

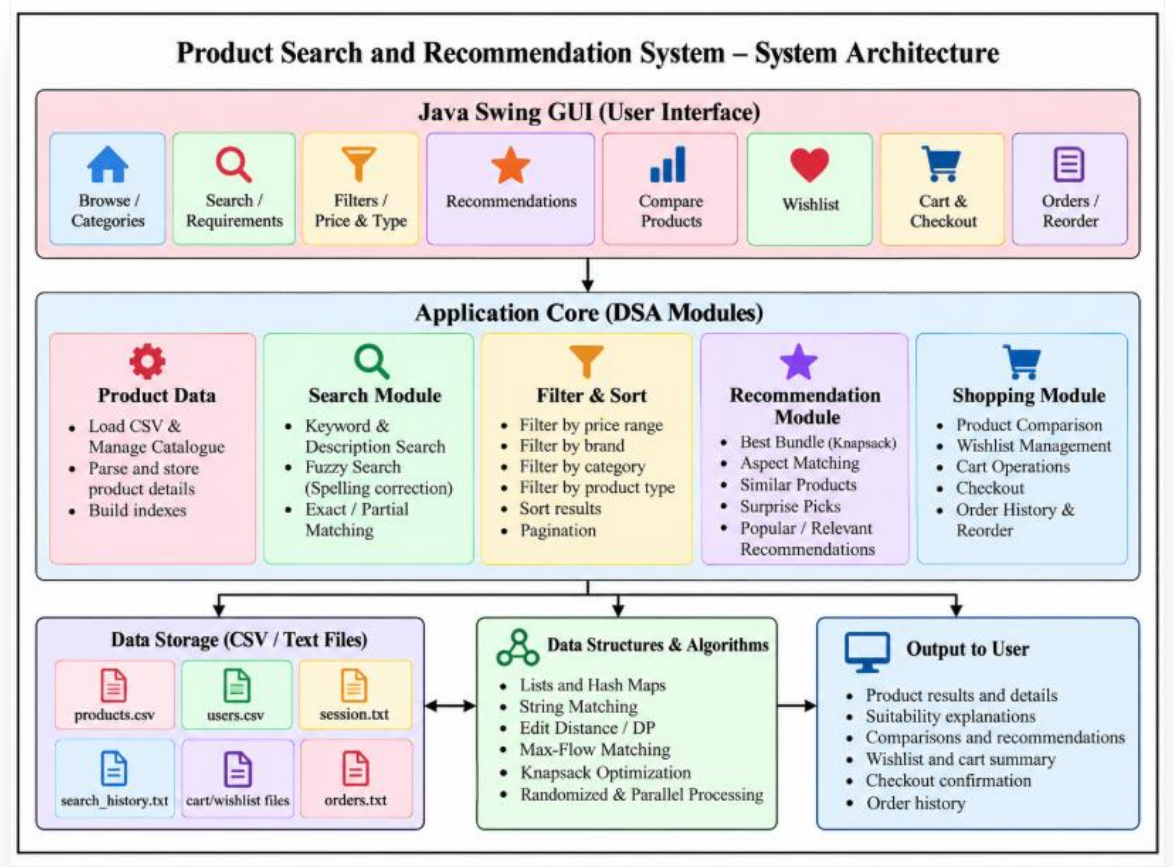


Fig. 1.1 Overview of the Product Search and Recommendation System

1.2 Problem Statement

Large shopping catalogues contain thousands of products with overlapping names, types, brands, features, and price ranges. A user may not know the exact product name and may instead describe a requirement such as “lightweight laptop with long battery life for students under 45000”. Exact string comparison is restrictive because the wording in the query may differ from the wording in the product description.

The project therefore addresses the problem of building a product-search and recommendation application that can process a large product dataset, accept natural-language-like requirements, search descriptions and product attributes, tolerate approximate wording, apply budget constraints, compare products, and explain why a recommended product is suitable.

1.3 Scope of the Project

The scope of the Product Search and Recommendation System for Shopping includes the following:

1. **Product Catalogue Management**: Loading and processing approximately 5,000 product records across 14 shopping categories.
2. **User Authentication:** Providing login, registration, and persistent session management.
3. **Product Browsing:** Displaying products by category with detailed product information.
4. **Requirement-Based Search:** Searching products using descriptions and user requirements through string-matching algorithms.
5. **Fuzzy Search and Filtering:** Supporting spelling correction, price filters, brand filters, category filters, and product sorting.
6. **Product Recommendations:** Providing budget-based bundles, aspect matching, similar products, and randomized recommendations.
7. **Explainable Recommendations:** Explaining product suitability using matching keywords, features, description similarity, and budget compatibility.
8. **Product Comparison and Wishlist:** Enabling users to compare products and save preferred items for later.
9. **Cart and Order Management:** Supporting cart operations, checkout, order history, and reordering.
10. **DSA Integration and GUI:** Applying data structures and algorithms through a Java Swing interface, with file-based storage for product and user-related information.

1.4 Objectives

The main objectives of the Product Search and Recommendation System for Shopping are as follows:

1. **To develop a Java-based shopping application** that enables users to search for, explore, and select products efficiently.
2. **To implement product catalogue management** for organizing and displaying products across multiple shopping categories.
3. **To provide requirement-based product search** that identifies suitable products using user descriptions and specific needs.
4. **To implement efficient string-matching algorithms** for keyword searching, description matching, and product retrieval.
5. **To support fuzzy searching and similarity calculation** to identify relevant products even when search queries contain spelling mistakes or variations.
6. **To develop intelligent product recommendation techniques** that suggest suitable products based on user requirements, preferences, and budget.
7. **To implement budget-based optimization and aspect matching** to help users identify appropriate products and combinations that satisfy their requirements.
8. **To provide explainable product recommendations** by displaying matching features, relevant keywords, description similarity, and budget compatibility.
9. **To integrate product comparison, wishlist, cart, and order management** to provide a convenient and complete shopping experience.
10. **To apply data structures and algorithms through a Java Swing graphical interface** while using file handling for persistent storage of product and user-related information.

CHAPTER 2 - SYSTEM REQUIREMENTS

2.1 Functional Requirements

2.1.1 Product Dataset Loading

The system shall load product records from data/products.csv during initialization and make the records available to search and recommendation operations.

2.1.2 Product Record Parsing

Each product record shall provide id, name, brand, category, type, price, and description information.

2.1.3 Product Storage

Parsed products shall be stored in Java data structures and accessed through ProductRepository.

2.1.4 Category Selection

The GUI shall allow users to select 14 shopping categories or All Products.

2.1.5 Product Browsing

The Browse tab shall display product ID, name, type, and price and allow details/add-to-cart actions.

2.1.6 Requirement Search

The Search tab shall accept a textual requirement and search relevant product information, especially descriptions and searchable attributes.

2.1.7 Price Filtering

The search interface shall accept minimum and maximum price values.

2.1.8 Pattern Matching

The system shall provide Naive, KMP, Z-function, and Rabin-Karp based string operations through PatternSearch.

2.1.9 Fuzzy Search

The system shall provide edit-distance based spelling correction and suggestions.

2.1.10 Similarity Search

The system shall calculate similarity using suffix-array/LCP and sequence-alignment techniques where appropriate.

2.1.11 Requirement Explanation

The system shall explain why a product is suitable by reporting matching evidence and budget/category compatibility.

2.1.12 Budget Recommendation

The system shall support budget-aware bundle selection using the knapsack routine.

2.1.13 Aspect Matching

The system shall match requirement aspects to candidate products using bipartite matching.

2.1.14 Surprise Recommendation

The system shall provide randomized category-based product selections.

2.1.15 Similar Product Recommendation

The system shall find similar products using description similarity, type, brand, and price-related signals.

2.1.16 Cart Management

The system shall support adding products, deleting products, displaying quantities, calculating totals, and saving cart state in user-specific files.

2.1.17 Wishlist, Comparison, Checkout and Orders

The system shall support a wishlist, compare two to four products by attributes and price, create an order record at checkout, clear the cart after successful checkout, display order history, and allow a previous order to be reordered.

2.1.18 Session Handling

The system shall save and read the active user from session.txt.

2.1.19 Input Validation

The system shall handle empty queries, invalid product IDs, invalid budgets, and invalid login information without unexpected termination.

2.1.20 Java Swing Interface

The system shall provide the shopping workflow through a Java Swing GUI.

2.2 Non-Functional Requirements

2.2.1 Usability

The GUI should provide a simple shopping-oriented interface without requiring users to understand the internal algorithms.

2.2.2 Performance

The catalogue should be loaded once and reusable data structures should reduce repeated file processing.

2.2.3 Scalability

The architecture should allow the product dataset to grow without changing the overall application workflow.

2.2.4 Reliability

Invalid input and missing resources should be handled through controlled messages.

2.2.5 Maintainability

Core DSA classes and GUI code should remain modular and independently understandable.

2.2.6 Portability

The project should run using a compatible JDK without a database or external library.

2.2.7 Data Consistency

The CSV record structure should remain consistent so the repository can parse each product correctly.

2.2.8 Error Handling

Search, cart, authentication, and checkout operations should respond clearly to invalid conditions.

2.2.9 Extensibility

New search, recommendation, and GUI features should be addable without redesigning the complete application.

2.3 Hardware and Software Requirements

Item	Requirement
Programming Language	Java
GUI Technology	Java Swing
IDE	Visual Studio Code
Data Storage	CSV and text files
Database	Not used
External Libraries	None
Compilation	javac -d bin src*.java
GUI Execution	java -cp bin SmartShoppingGUI
Console Execution	java -cp bin Main

2.4 Dataset Requirements

The current data/products.csv contains 5,000 product records. The dataset covers 14 categories, 142 product types and 590 brands. Product prices in the current file range from Rs 39 to Rs 251,999.

Field	Description
id	Unique product identifier
name	Product name
brand	Brand
category	Shopping category
type	Product type
price	Product price in rupees
description	Detailed text used for search and recommendation evidence

Role of the Dataset

The dataset serves as the primary source of product information for the application. The system loads the records and uses their attributes to display products, process search queries, apply filters, compare alternatives, and generate recommendations.

Data Storage

The application uses file-based storage to manage product and user-related information. Product details are maintained in products.csv, while user accounts, sessions, search history, cart information, wishlist items, and orders are managed through the corresponding CSV or text files supported by the implementation.

CHAPTER 3 - SYSTEM DESIGN

3.1 High-Level Architecture

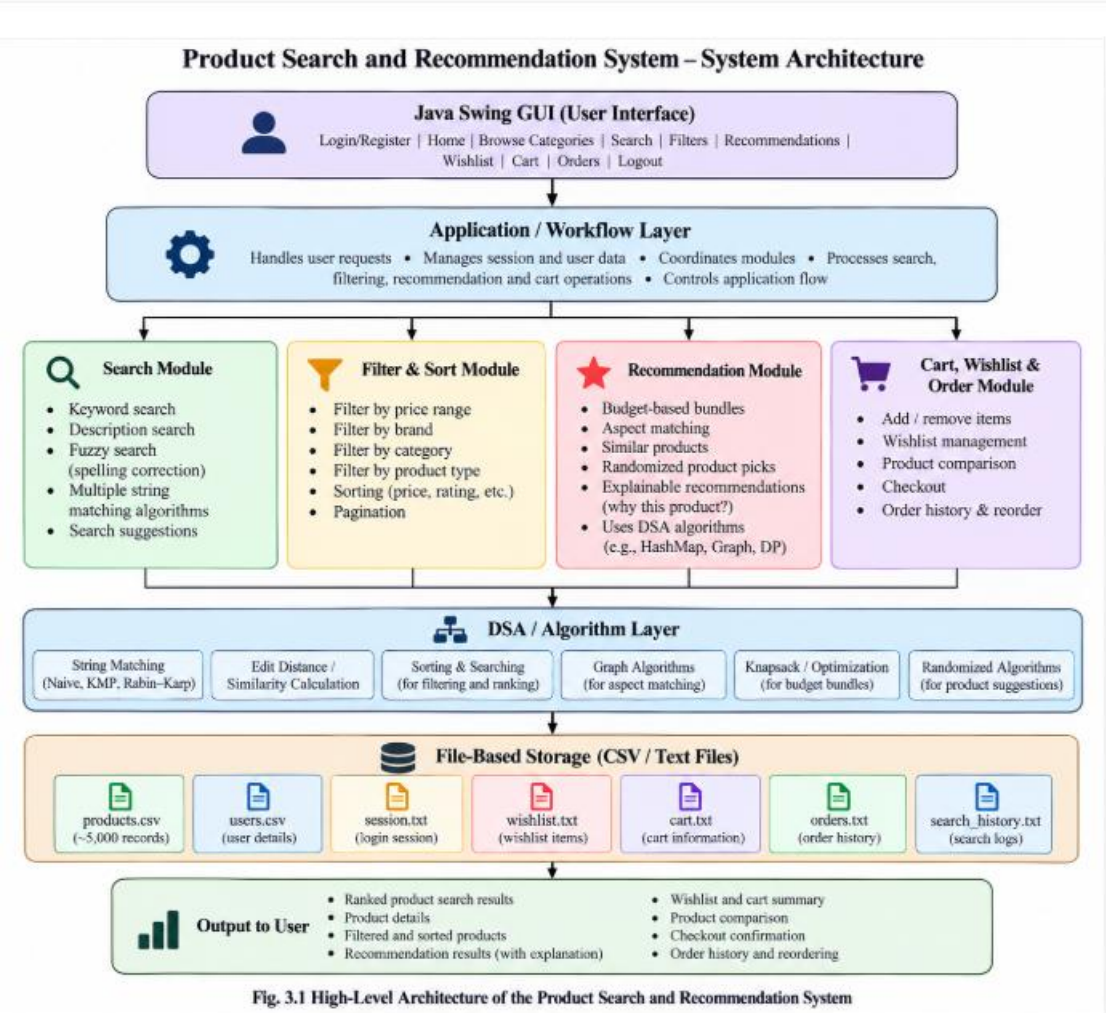


Fig. 3.1 System Architecture of the Product Search and Recommendation System

The architecture separates the data layer, repository, DSA processing, recommendation logic, and Java Swing user interface. ProductRepository loads the product CSV. SearchEngine combines the algorithmic modules to produce ranked search hits and explainability information. The GUI calls the core classes rather than duplicating their algorithms.

3.2 System Flowchart

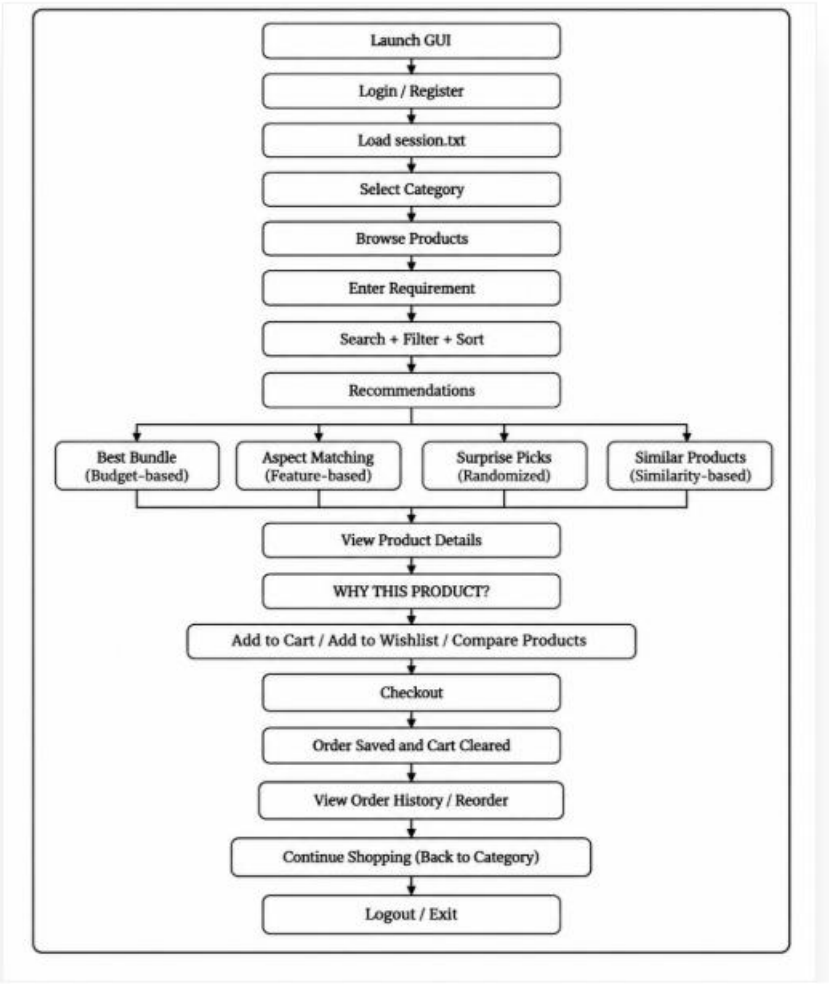


Fig. 3.2 System Flowchart

The system begins with GUI launch and session checking. A user can login/register, choose a category, browse products, enter a requirement, search/filter the catalogue, open recommendations, inspect product details, add products to the cart, and checkout or continue shopping.

3.3 Data and Component Description

Component	Main Responsibility
SmartShoppingGUI.java	Java Swing interaction layer; login, browsing, search, recommendations, cart and checkout.
ProductRepository.java	Loads products.csv, stores products and builds the keyword index.
SearchEngine.java	Combines search signals, scoring, diversification and explainability.
PatternSearch.java	Naive, KMP, Z-function and Rabin-Karp string matching.
SimilaritySearch.java	Suffix array, Kasai LCP, longest common substring, Needleman-Wunsch and Smith-Waterman.
FuzzySearch.java	Wagner-Fischer edit distance, Damerau distance and spelling suggestions.

NetworkFlow.java	Edmonds-Karp maximum flow, min-cut and bipartite matching.
GreedyRecommendation.java	Knapsack, Optimal BST, TSP, 2-approximate vertex cover and exact vertex cover.
ParallelSearch.java	Miller-Rabin, randomized quicksort, reservoir sampling, parallel prefix sum/reduce.
ProblemClassifier.java	Maps query characteristics to problem classes and algorithm families.
Auth.java	Registration, login, password hashing and persistent session handling.
Cart.java	Cart IDs, quantities, totals and cart-file persistence.

3.4 DSA and CO Mapping

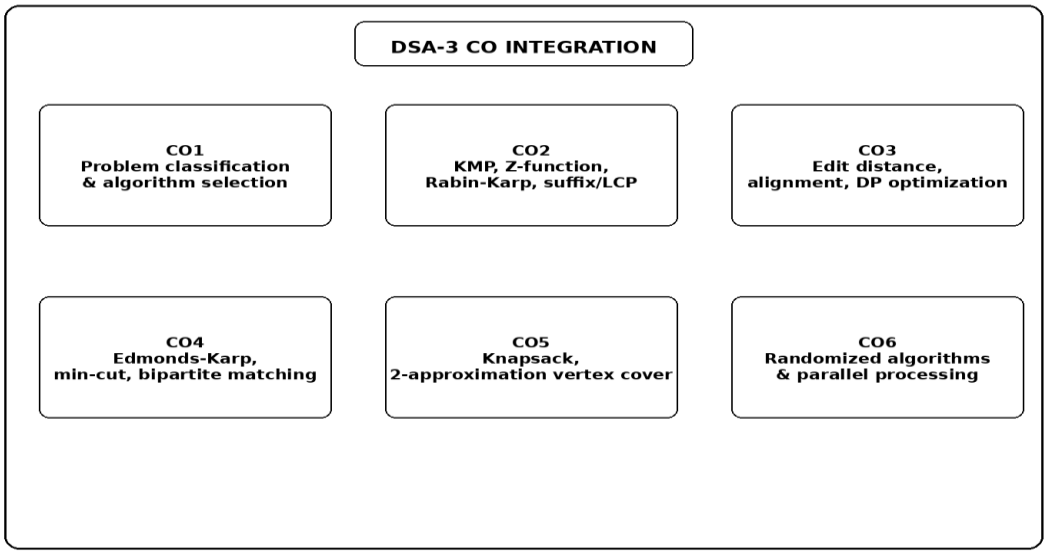


Fig. 3.3 DSA and CO Mapping

CO	**Implementation**	**Application Use**
CO1	ProblemClassifier	Identifies query/problem characteristics and suggests an algorithm family.
CO2	PatternSearch + SimilaritySearch	KMP, Z-function, Rabin-Karp, suffix array, LCP and string similarity for product search.
CO3	FuzzySearch + optimization	Edit distance, Damerau distance, sequence alignment and budget optimization.
CO4	NetworkFlow	Maximum flow, min-cut and bipartite matching for requirement-aspect assignment.
CO5	GreedyRecommendation	Knapsack and 2-approximation vertex cover for optimization demonstrations.
CO6	ParallelSearch	Randomization, Miller-Rabin, randomized quicksort, reservoir sampling and parallel processing.

3.5 Dataset Design

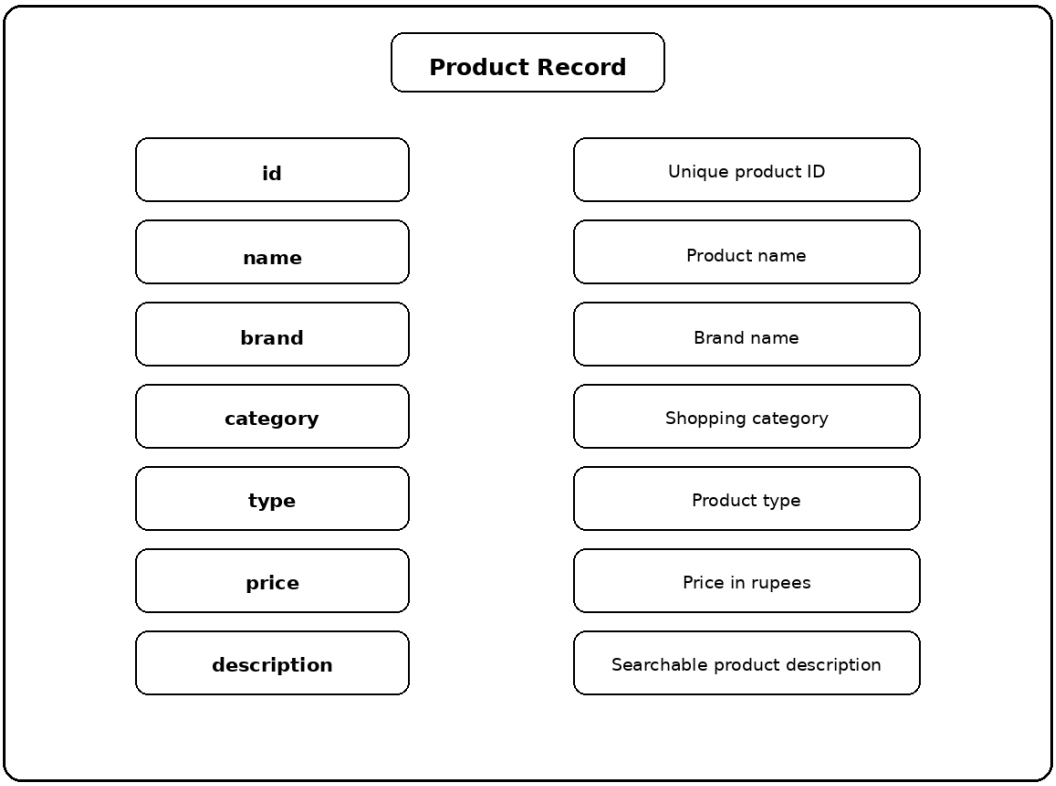


Fig. 3.4 Product Dataset Record Structure

The product record is represented by seven logical fields. The description is particularly important because requirement-based searching uses description text together with name, brand, category and type signals. The dataset is kept as a CSV file so that the complete project remains portable and editable in VS Code.

CHAPTER 4 - IMPLEMENTATION

4.1.1 Product Loading

ProductRepository.load reads products.csv, creates Product objects, records categories, and prepares the inverted keyword index.

4.1.2 Product Search

SearchEngine accepts a requirement, extracts keywords/budget, obtains candidate products and computes relevance scores.

4.1.3 Naive Pattern Matching

PatternSearch.naiveSearch performs direct pattern comparisons.

4.1.4 KMP

PatternSearch builds the LPS array and uses KMP search/count operations.

4.1.5 Z-Function

PatternSearch computes the Z array and supports prefix-oriented product-name matching.

4.1.6 Rabin-Karp

PatternSearch uses rolling hash and verification to locate patterns.

4.1.7 Suffix Array and LCP

SimilaritySearch builds suffix-array information and Kasai LCP values.

4.1.8 Sequence Alignment

Needleman-Wunsch and Smith-Waterman provide global/local alignment scores.

4.1.9 Fuzzy Search

FuzzySearch implements Wagner-Fischer edit distance and Damerau distance and can suggest corrected query wording.

4.1.10 Bipartite Matching

NetworkFlow obtains aspect-to-product matching through maximum-flow based matching.

4.1.11 Budget Optimization

GreedyRecommendation.knapsackPick selects a high-value product bundle within the requested budget.

4.1.12 Random Recommendation

ParallelSearch.reservoirSample selects a bounded set of category products.

4.1.13 Similar Product Search

The GUI combines description similarity, type/brand matches and price closeness, then ranks candidates using randomized quicksort.

4.1.14 Cart and Checkout

Cart stores product IDs and quantities, calculates totals, and checkout creates an order record.

4.1.15 Java Swing GUI

SmartShoppingGUI provides Login/Register, Browse, Search, Recommendations, Cart, category selection, product details, and checkout.

4.2 PSEUDOCODE

Product Search

```
ALGORITHM SearchProducts(query, category, minPrice, maxPrice)

    IF query is empty THEN
        RETURN "Enter a valid search query"
    END IF

    keywords ← ExtractKeywords(query)
    budget ← ExtractBudget(query)

    IF spelling errors are detected THEN
        keywords ← FuzzyCorrection(keywords)
    END IF

    candidates ← RetrieveProducts(category)

    FOR each product IN candidates DO
        IF product.price < minPrice OR
           product.price > maxPrice THEN
            CONTINUE
        END IF

        matchScore ← PatternMatching(product, keywords)
        similarity ← DescriptionSimilarity(product, query)
        relevance ← CalculateScore(matchScore, similarity)

        IF relevance > 0 THEN
            Add product to results
        END IF
    END FOR

    Sort results by relevance
    RETURN ranked results

END ALGORITHM
```

SEARCH_PRODUCTS(query, category, minPrice, maxPrice)

1. Validate query.
2. Optionally request fuzzy correction.
3. Extract keywords and budget.
4. Retrieve candidates.
5. Apply category and price constraints.
6. Evaluate matching/similarity signals.
7. Compute relevance and suitability.
8. Diversify and return ranked hits.

Explain Product Suitability

```
ALGORITHM ExplainProduct(product, keywords, budget)

    matchingFeatures ← FindMatchingFeatures(product, keywords)
    similarity ← CalculateDescriptionSimilarity(
                    product.description, keywords)
    categoryMatch ← CheckCategoryCompatibility(product)
    budgetMatch ← CheckBudget(product.price, budget)

    score ← CalculateRelevance(
                matchingFeatures, similarity,
                categoryMatch, budgetMatch)

    IF score >= HIGH_THRESHOLD THEN
        suitability ← "YES"
    ELSE IF score >= LOW_THRESHOLD THEN
        suitability ← "PARTIALLY"
    ELSE
        suitability ← "NO"
    END IF

    DISPLAY product details
    DISPLAY suitability
    DISPLAY "WHY THIS PRODUCT?"
    DISPLAY matching features
    DISPLAY description similarity
    DISPLAY category and budget compatibility

END ALGORITHM
```

EXPLAIN_PRODUCT(hit, keywords)

1. Read product.
2. Identify matching keywords/features.
3. Calculate description similarity evidence.
4. Compare category/type and budget.
5. Produce YES, PARTIALLY, or NO.
6. Display “WHY THIS PRODUCT?” evidence.

Budget Bundle

```
ALGORITHM BudgetBundle(query, budget)

    products ← SearchRelevantProducts(query)

    FOR each product IN products DO
        weight[product] ← product.price
        value[product] ← product.relevanceScore
    END FOR

    n ← NumberOfProducts(products)
    INITIALIZE DP[0...n][0...budget] TO 0

    FOR i ← 1 TO n DO
        FOR b ← 0 TO budget DO
            IF weight[i] <= b THEN
                DP[i][b] ← MAX(
                    DP[i-1][b],
                    value[i] + DP[i-1][b-weight[i]]
                )
            ELSE
                DP[i][b] ← DP[i-1][b]
            END IF
        END FOR
    END FOR

    selectedProducts ← BacktrackSelectedProducts(DP)
    totalCost ← CalculateTotalPrice(selectedProducts)
    remainingBudget ← budget - totalCost

    DISPLAY selectedProducts
    DISPLAY totalCost
    DISPLAY remainingBudget

END ALGORITHM
```

BUDGET_BUNDLE(query, budget)

1. Search relevant products inside budget.
2. Convert relevance to value and price to weight.
3. Apply 0/1 knapsack.
4. Display selected bundle, total cost and remaining budget.

Aspect Matching

```
ALGORITHM AspectMatching(aspects)

    requirements ← SplitIntoAspects(aspects)
    products ← RetrieveCandidateProducts(requirements)

    CREATE flow network with:
        Source node
        Requirement nodes
        Product nodes
        Sink node

    FOR each requirement DO
        ADD edge from Source to Requirement
        SET capacity ← 1
    END FOR

    FOR each requirement DO
        FOR each product DO
            IF ProductMatchesAspect(product, requirement) THEN
                ADD edge from Requirement to Product
                SET capacity ← 1
            END IF
        END FOR
    END FOR

    FOR each product DO
        ADD edge from Product to Sink
        SET capacity ← 1
    END FOR

    maxFlow ← EdmondsKarp(flowNetwork)
    assignments ← ExtractMatchedProducts(flowNetwork)

    DISPLAY each requirement and assigned product
    DISPLAY maximum matching count

END ALGORITHM
```

ASPECT_MATCH(aspects)

1. Split requirement into aspects.
2. Search candidate products.
3. Create adjacency matrix.
4. Run bipartite matching through maximum flow.
5. Display each aspect and assigned product.

4.3 JAVA SWING INTERFACE

The Java Swing GUI is implemented in SmartShoppingGUI.java using standard Swing components and table models. The main tabs are Browse, Search, Recommendations, Wishlist, Cart, and Orders. Browse supports product details, adding to cart, wishlist actions, comparison of two to four products, sorting, pagination, and configurable rows per page. Search accepts requirement text and price limits and displays result count, elapsed search time, keywords, scores, suitability, and recent searches. Recommendations contains Best Bundle (knapsack), Aspect Matching, Surprise Picks, and Similar Products. Cart supports adding and deleting items, totals, checkout, and order persistence; Orders shows previous order records and provides a reorder action.

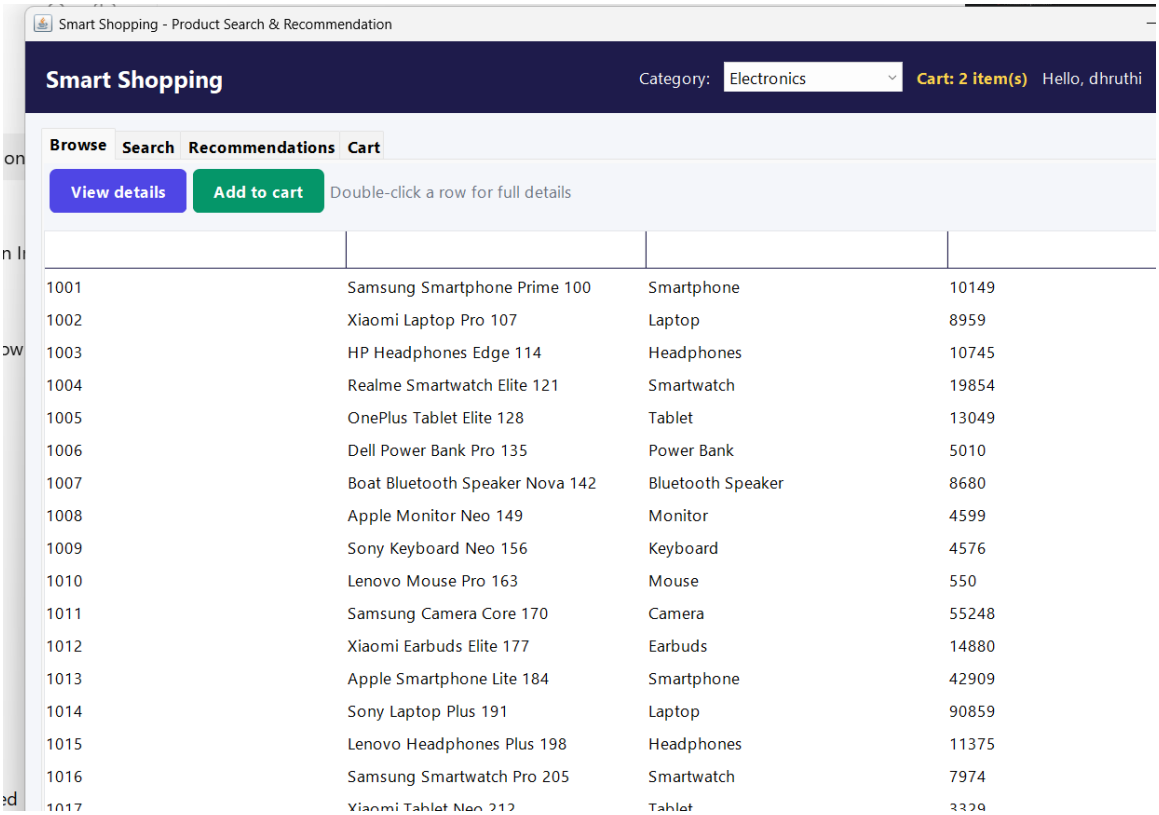


Fig. 4.1 Java Swing Shopping Dashboard

CHAPTER 5 - RESULTS

5.1 Testing Strategy

Test Input / Action	Expected Result	Observed Result
Launch GUI	Shopping GUI opens	GUI opens through SmartShoppingGUI
Select Electronics	Electronics products displayed	Category browsing supported
gaming laptop under 50000	Relevant products within budget	7 products shown in recorded test
Price filter	Products restricted by range	Minimum/maximum fields supported
Product details	Full product information	Details dialog available
WHY THIS PRODUCT?	Evidence-based explanation	Matching keywords, similarity, category/type and budget shown
Best bundle	Budget-aware bundle	Knapsack implementation connected
Aspect matching	Aspect-to-product assignment	Network-flow matching connected
Surprise picks	Random products	Reservoir sampling connected
Similar products	Similar results	Similarity + randomized sorting connected
Cart	Add/delete/total	Cart module connected
Checkout	Order saved and cart cleared	Checkout writes orders.txt
Product comparison	Selected products compared by attributes and price	Comparison window displays product details and price differences
Wishlist	Products can be added to and removed from the wishlist	Wishlist management is supported
Search history	Previous searches can be viewed and selected again	Recent searches are displayed in the Search interface
Pagination and sorting	Products can be viewed across pages and sorted	Pagination controls and sorting options are available
Order history and reorder	Previous orders are displayed and selected products can be reordered	Order history and reorder functionality are available

5.2 Functional Results

5.2.1 Product Catalogue Loading

ID	Name	Brand	Type	Price (Rs)
1001	Sennheiser Headphones Studi...	Sennheiser	Headphones	6199
1002	Dell Power Bank 5000mAh Poc...	Dell	Power Bank	1699
1003	Philips Bluetooth Speaker Out...	Philips	Bluetooth Speaker	789
1004	Samsung Monitor 27 inch QH...	Samsung	Monitor	18399
1005	Razer Keyboard Wireless Slim	Razer	Keyboard	2299
1006	Zebronics Mouse Gaming 120...	Zebronics	Mouse	3699
1007	Nothing Phone 2 256GB Smar...	Nothing	Smartphone	50399
1008	Lenovo IdeaPad Gaming 3 16...	Lenovo	Laptop	74699
1009	Noise ColorFit Icon 2 Silver Sm...	Noise	Smartwatch	1799
1010	Realme Pad 2 WiFi 128GB Tabl...	Realme	Tablet	20699
1011	Canon EOS 200D II Combo Kit...	Canon	Camera	68399
1012	Samsung Galaxy Buds3 Pro W...	Samsung	Earbuds	19999
1013	boAt Headphones Kids Safe V...	boAt	Headphones	1099
1014	Anker Power Bank 30000mAh ...	Anker	Power Bank	1799
1015	Anker Bluetooth Speaker Hom...	Anker	Bluetooth Speaker	1399

5.2.2 Product Browsing and Category Selection

The Browse tab allows the user to select a product category and view the available products with their ID, name, type, and price. The interface also provides options to view complete product details and add a selected product to the cart.

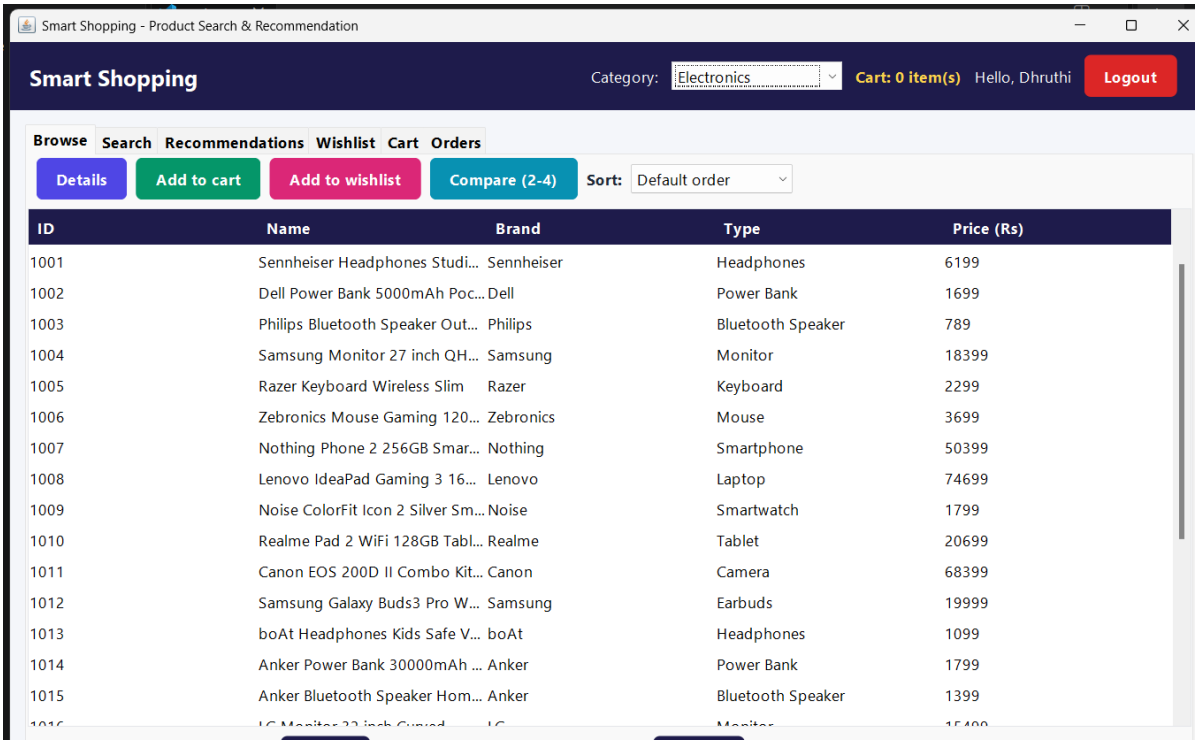


Fig. 5.1 Product Browsing and Category Selection

5.2.3 Requirement-Based Product Search

The Search tab accepts a free-text requirement and optional minimum and maximum prices. In the captured test, the query “i want a smartphone” returned 14 results in 22 ms, showing a keyword summary, relevance scores, and a suitability indicator. A recent-search selector and Clear history control are also visible.

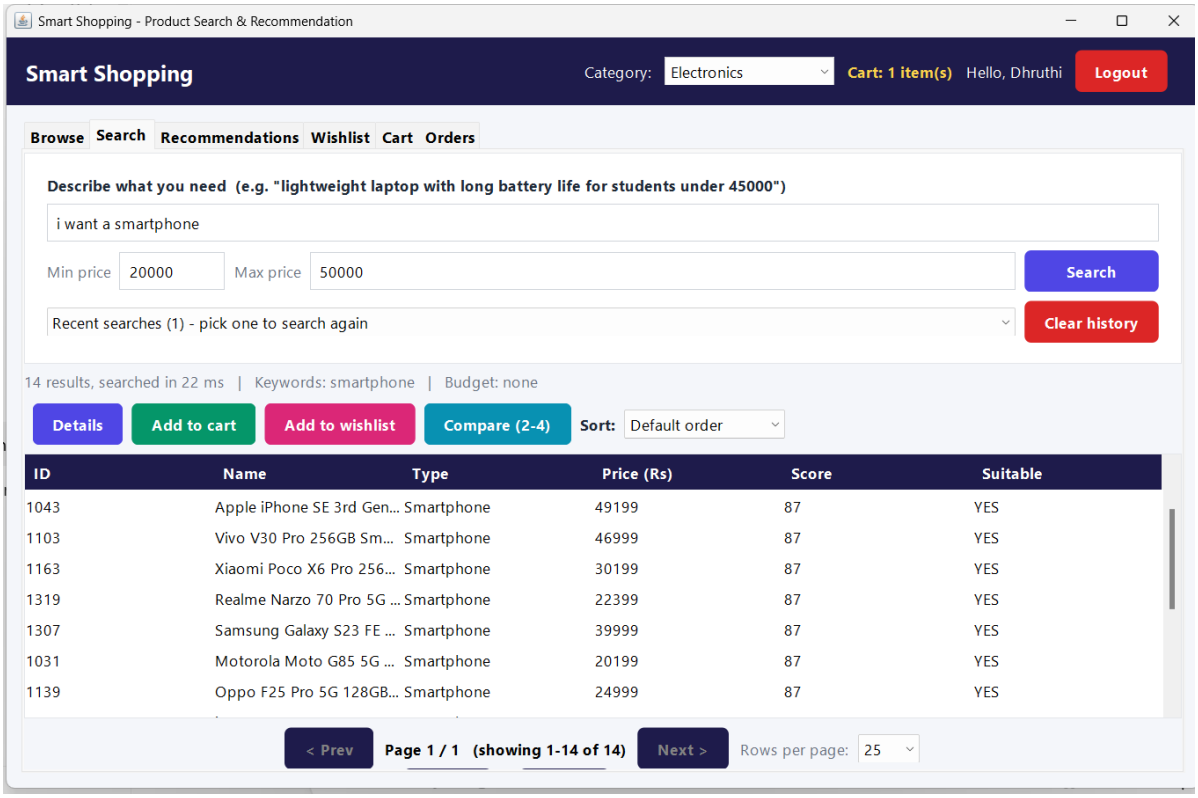


Fig. 5.2 Requirement-Based Product Search

5.2.4 Product Details and Explainable Recommendation

The Product Details dialog displays the selected product's ID, name, brand, category, type, price, and complete description. It provides direct actions to add the product to the cart or wishlist, allowing users to continue shopping without leaving the browse view.

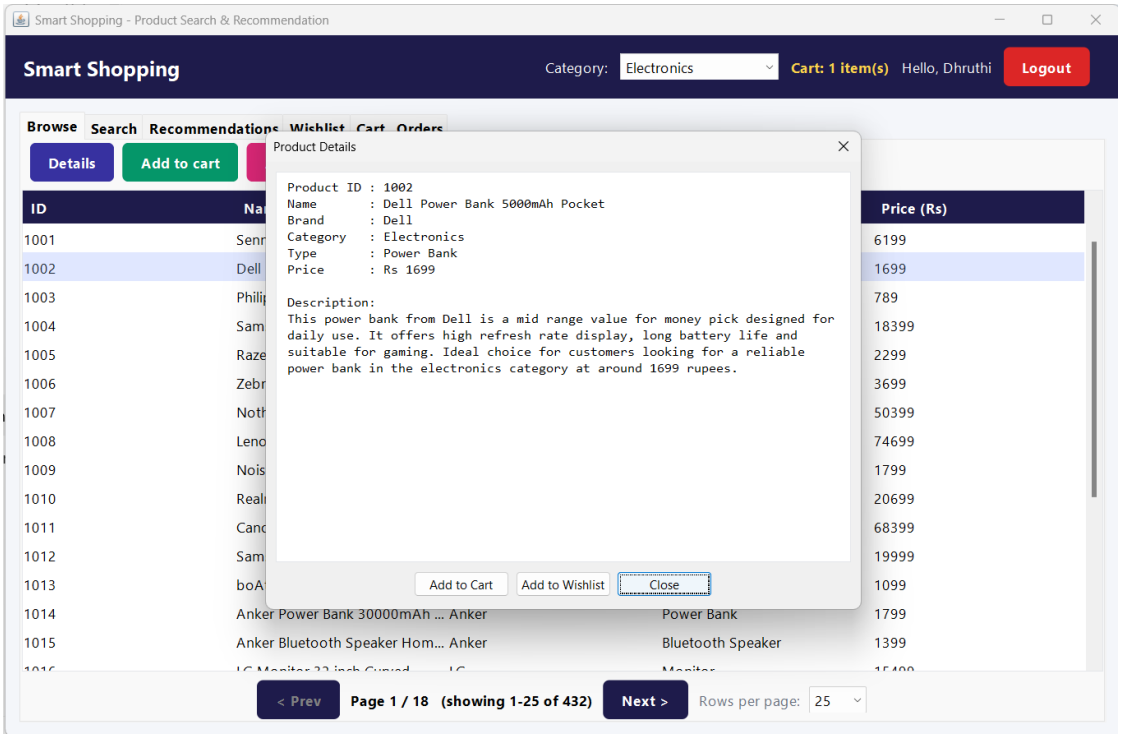


Fig. 5.3 Product Details and Explainable Recommendation

5.2.5 Budget-Based Recommendation

The Best Bundle feature applies a budget constraint and selects a combination of relevant products using the knapsack-based optimization module. The displayed result shows the selected products, total price, remaining budget, and total relevance achieved.

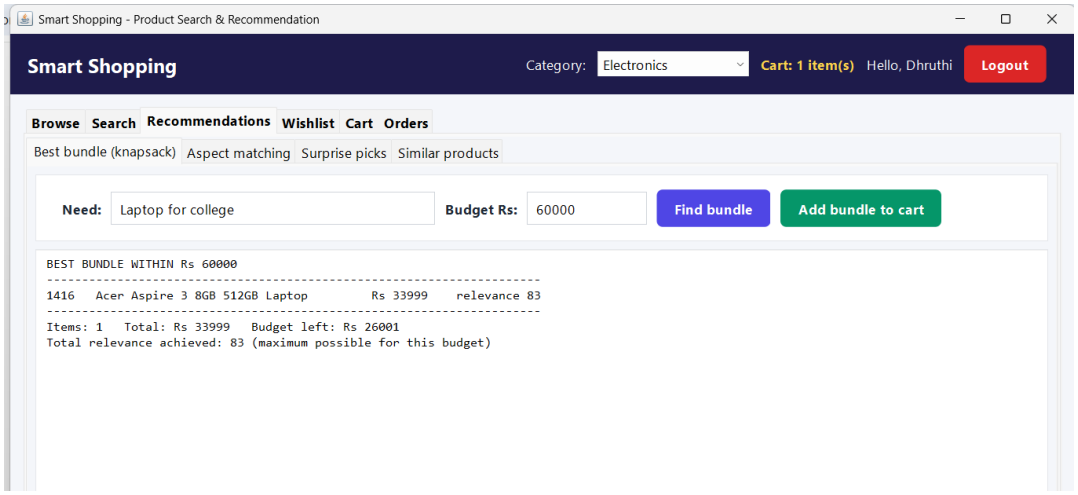


Fig. 5.4 Budget-Based Product Recommendation using Knapsack

5.2.6 Aspect-Based Product Matching

The Aspect Matching feature accepts multiple product requirements and attempts to assign each aspect to a distinct suitable product. The result reports successful assignments, unmatched aspects, and the maximum matching size obtained through the graph-based matching module.

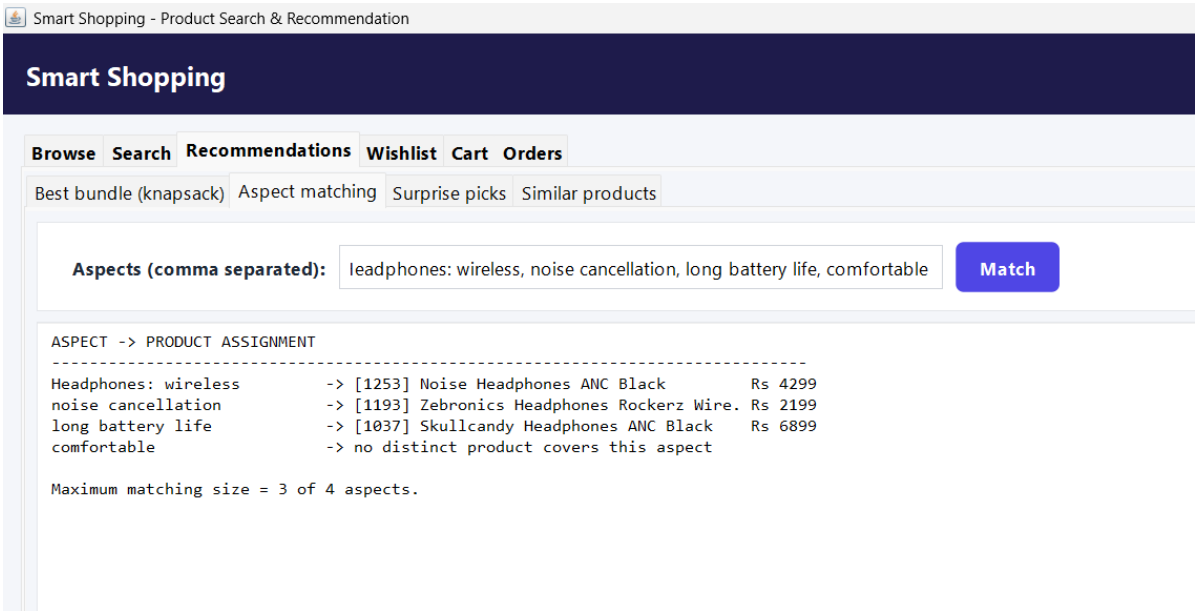


Fig. 5.5 Aspect-Based Product Matching

5.2.7 Randomized Product Recommendation

The Surprise Picks feature uses randomized selection to present a set of products from the selected category. This supports exploratory shopping and demonstrates the use of randomized algorithms within the application.

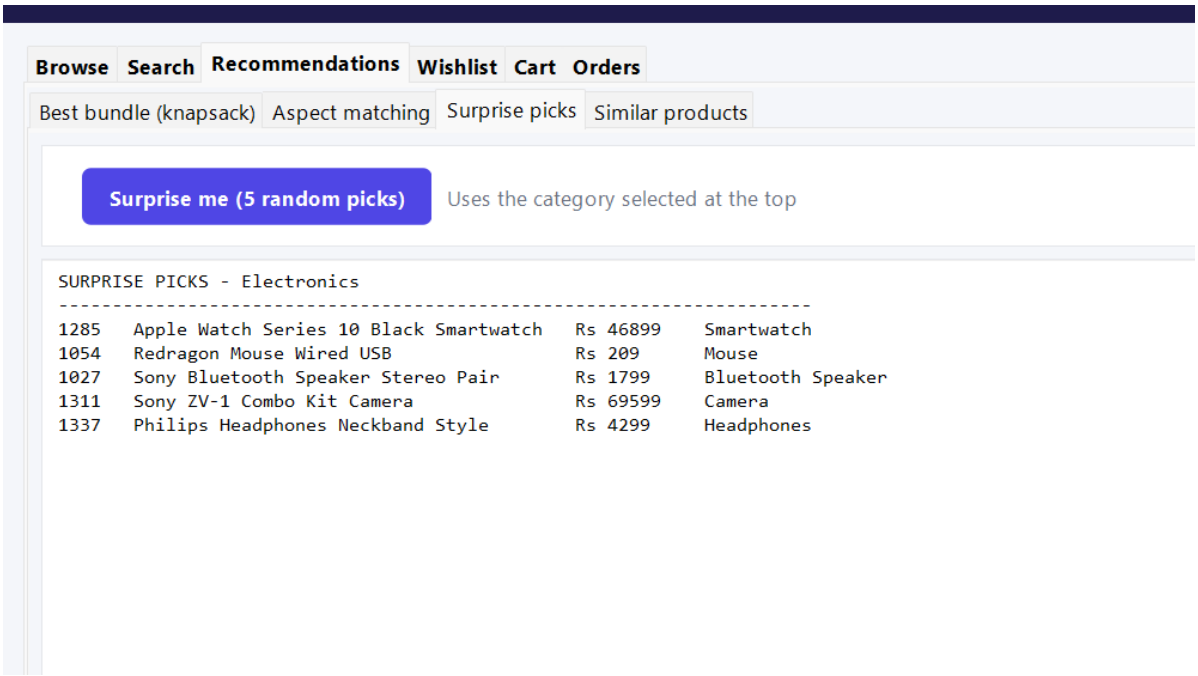


Fig. 5.6 Randomized Product Recommendation

5.2.8 Similar Product Recommendation

The Similar Products feature takes a selected product ID and returns other products with high similarity scores. The output demonstrates similarity-based recommendation using product characteristics and text-related signals.

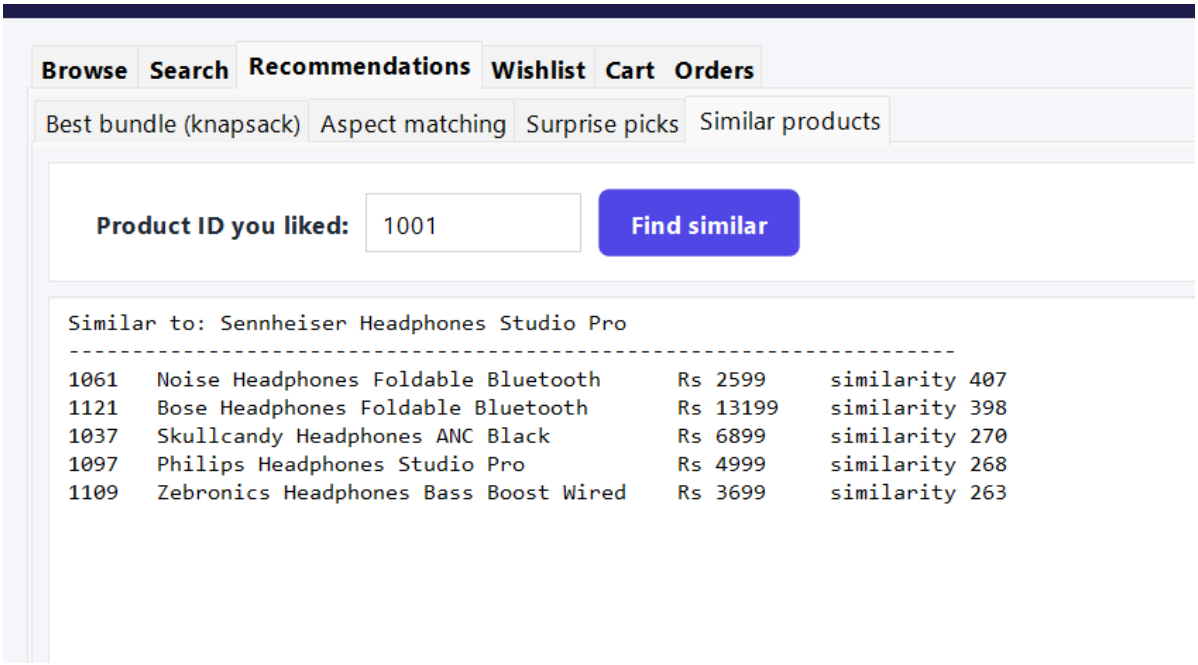


Fig. 5.7 Similar Product Recommendation

5.2.9 Shopping Cart and Checkout

The Cart tab displays product IDs, names, quantities, unit prices, and subtotals. Users can add products by ID, delete selected items, and check out. The captured cart summary shows the total item count and price, with an additional total verification using parallel reduction and a shortest store-pickup route summary.

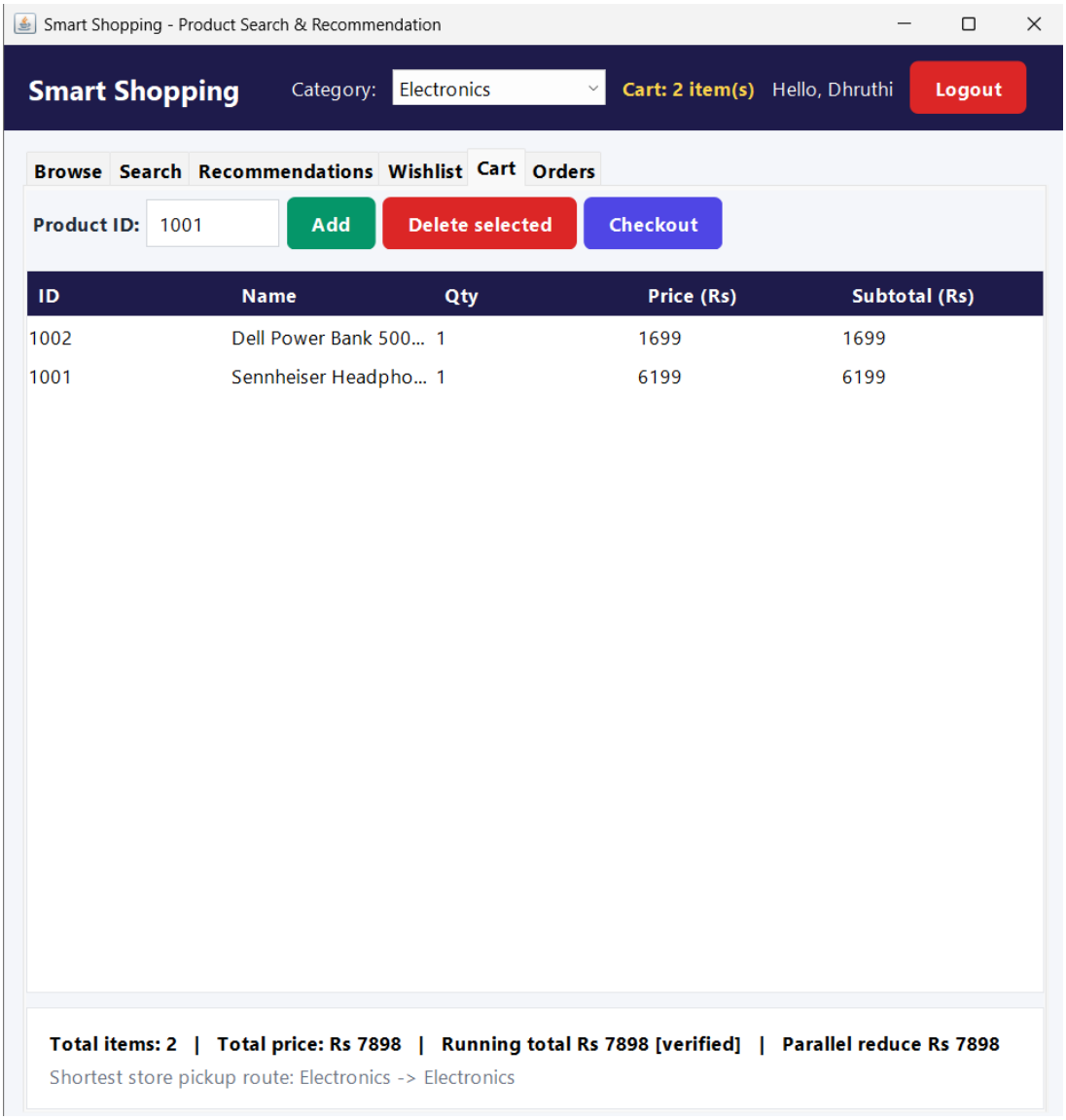


Fig. 5.8 Shopping Cart and Checkout

5.2.10 Product Comparison

The Compare (2–4) action presents selected products side by side, including name, brand, category, type, price, price difference from the least expensive option, key features, and similarity information. This view helps users compare alternatives before choosing a product.

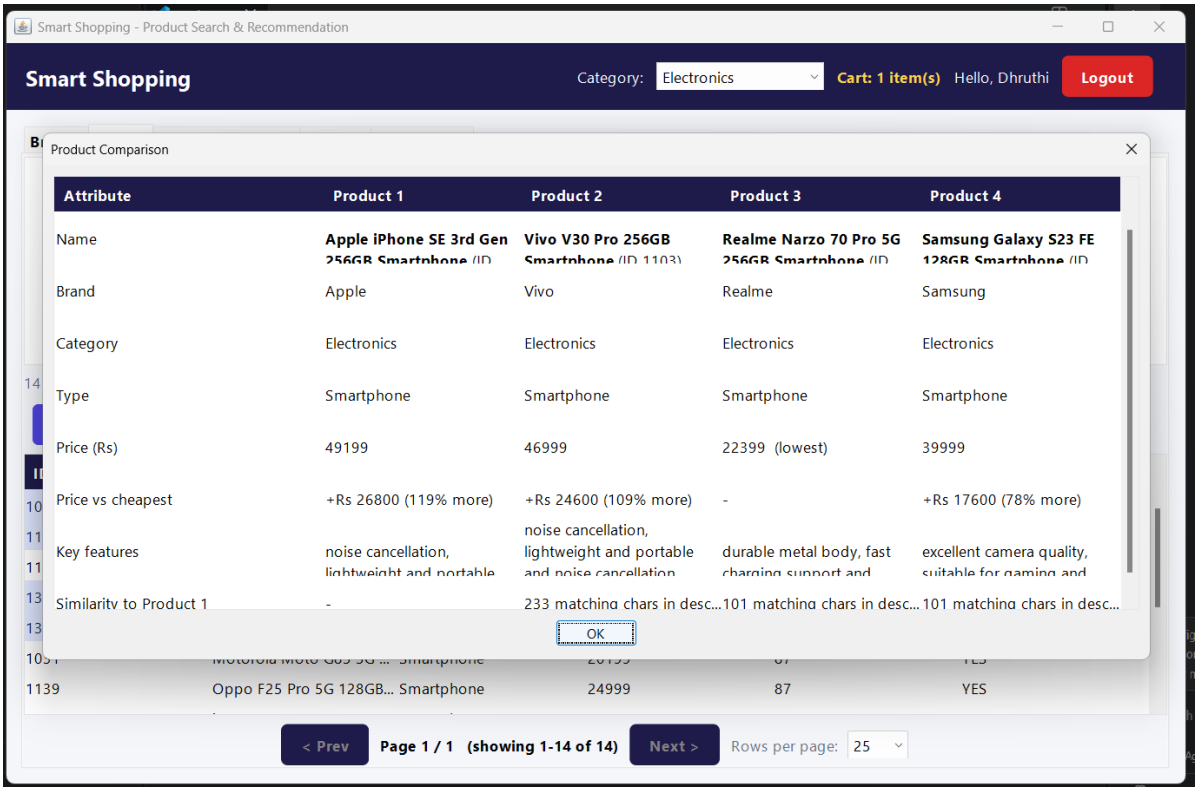


Fig. 5.9 Product Comparison of Selected Products

5.2.11 Wishlist Management

The Wishlist tab stores products saved for later review. Users can inspect saved products, open details, add a saved item to the cart, remove items from the wishlist, and compare selected products.

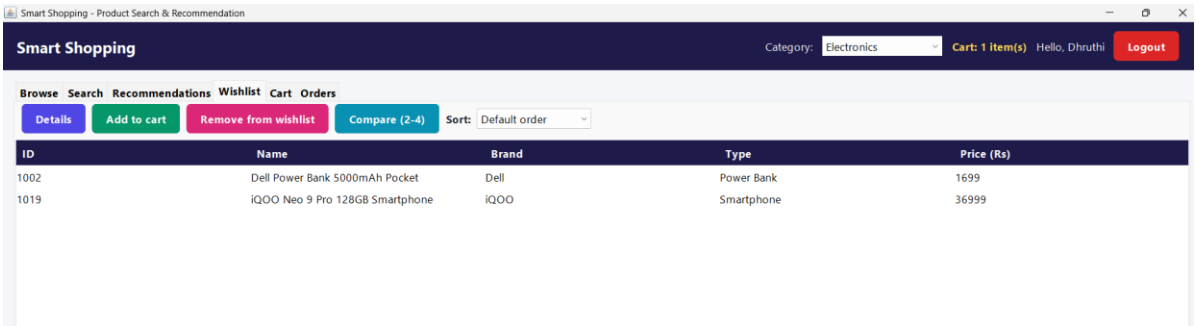


Fig. 5.10 Wishlist Management

5.2.12 Checkout Confirmation

After checkout, the application displays an order confirmation with an order ID, item count, amount, customer name, and confirmation that the order was saved to data/orders.txt and the cart was cleared.

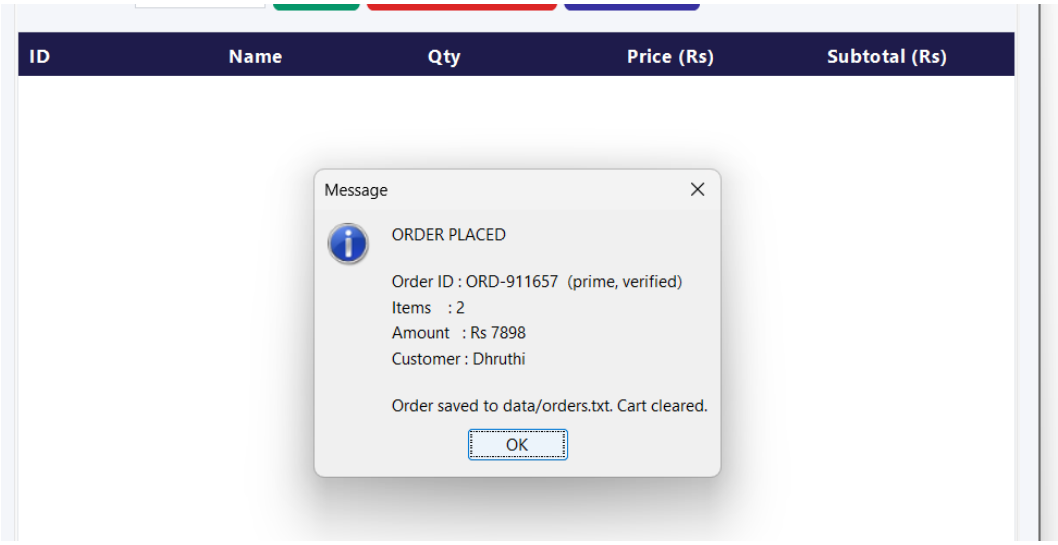


Fig. 5.11 Checkout Confirmation

5.2.13 Order History and Reordering

The Orders tab displays previous order IDs, dates, item counts, and amounts. The interface provides sorting options and a Reorder selected action to support repeat shopping.

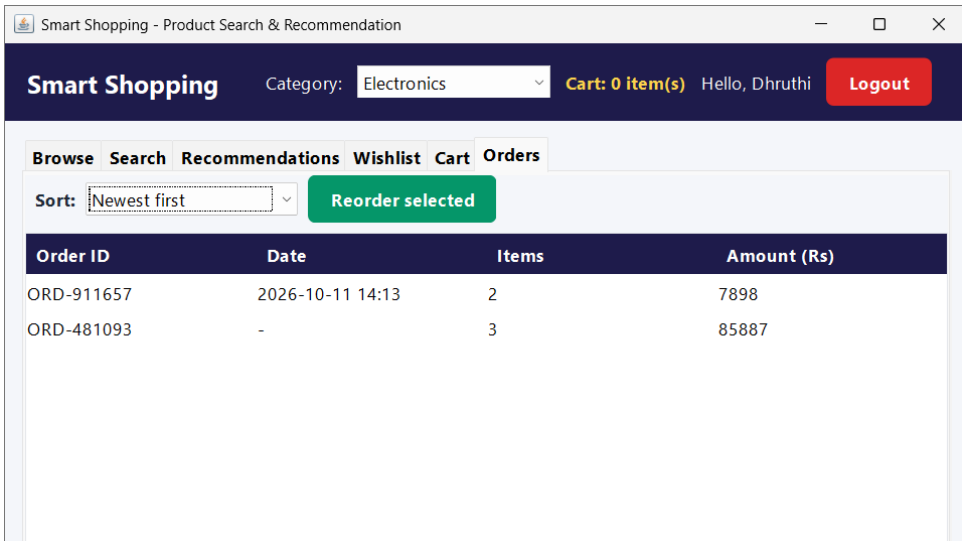


Fig. 5.12 Order History and Reordering

CHAPTER 6 — CHALLENGES & LIMITATIONS

6.1.1 Integrating Multiple Search Algorithms

Different string algorithms have different inputs and matching strategies. The project combines them through SearchEngine so the GUI remains simple.

6.1.2 Handling a Large Product Catalogue

A 5,000-record CSV requires initialization and indexing so queries do not repeatedly perform expensive file parsing.

6.1.3 Requirement Interpretation

Users describe products using multiple words, features and budgets instead of exact names.

6.1.4 Fuzzy Search

Spelling variations and wording differences require edit-distance based correction.

6.1.5 Explainable Recommendations

The system must identify the evidence behind a recommendation rather than only outputting a score.

6.1.6 Requirement-Aspect Matching

Multiple requirement aspects may need distinct products, creating a graph matching problem.

6.1.7 Budget Optimization

Selecting relevant products under a fixed budget is an optimization problem.

6.1.8 GUI Integration

Existing DSA modules had to be connected to Swing without duplicating algorithm logic.

6.2 Solutions Applied

6.2.1 Search Integration

SearchEngine combines multiple matching signals before producing ranked results.

6.2.2 Catalogue Preparation

ProductRepository loads the catalogue and constructs a keyword index.

6.2.3 Fuzzy Correction

FuzzySearch provides edit-distance and suggestion support.

6.2.4 Explainability

SearchEngine.explain reports matched keywords/features, description similarity, category/type and budget evidence.

6.2.5 Graph Matching

NetworkFlow provides bipartite matching and maximum-flow operations.

6.2.6 Optimization

GreedyRecommendation provides knapsack and additional optimization routines.

6.2.7 Randomization and Parallelism

ParallelSearch provides reservoir sampling, randomized quicksort, Miller-Rabin, prefix sum and reduction.

6.2.8 GUI Separation

SmartShoppingGUI remains a separate front-end class and calls the existing DSA/business classes.

6.3 Limitations

- Product information is stored in CSV/text files rather than a production database.
- The dataset provenance and commercial pricing accuracy are not established as live marketplace data; the catalogue is a project dataset containing real-world-style brands and categories.
- Recommendation quality depends on the text available in product descriptions and the implemented matching signals.
- Parallel processing may not improve every workload because thread-management overhead can outweigh the benefit for small inputs.
- Advanced algorithms are integrated for course demonstration and their practical contribution depends on the query type.
- The application is a Java Swing desktop application and does not provide a web/mobile deployment.
- Cart/order persistence is file-based and intended for an academic project environment.

CHAPTER 7 — CONCLUSION & FUTURE WORK

The Product Search and Recommendation System for Shopping demonstrates the practical application of Data Structures and Algorithms to searching, matching, comparing, optimizing, and recommending products from a large catalogue. The project is implemented in Java and currently operates on a 5,000-record product dataset stored in data/products.csv.

The system combines Naive Pattern Matching, KMP, Z-function, Rabin-Karp, suffix-array/LCP processing, edit distance, sequence alignment, network flow, bipartite matching, knapsack optimization, approximation, randomized algorithms and parallel processing. These techniques are connected to actual shopping functions rather than being presented as isolated demonstrations.

The Java Swing interface presents product browsing, requirement search, filters, recommendations, product details, explainability, cart management and checkout as normal user operations. The internal DSA algorithms remain separated from the GUI, allowing the interface and algorithm implementations to be maintained independently.

The explainable recommendation mechanism provides a direct connection between algorithmic processing and the user's shopping decision by identifying matching keywords/features, description similarity, category/type compatibility and budget fit.

7.2 FUTURE WORK

7.2.1 Improved Search Ranking

Use richer semantic representations and improved ranking while retaining the DSA engine.

7.2.2 Personalized Recommendations

Use user search and purchase history to personalize ranking.

7.2.3 Real-Time Product Information

Integrate live price, inventory, ratings and availability from authorized sources.

7.2.4 Larger Verified Datasets

Use documented and licensed product datasets with clear provenance.

7.2.5 Advanced Natural-Language Understanding

Improve extraction of product type, features, use case and budget from free-form requirements.

7.2.6 Web or Mobile Interface

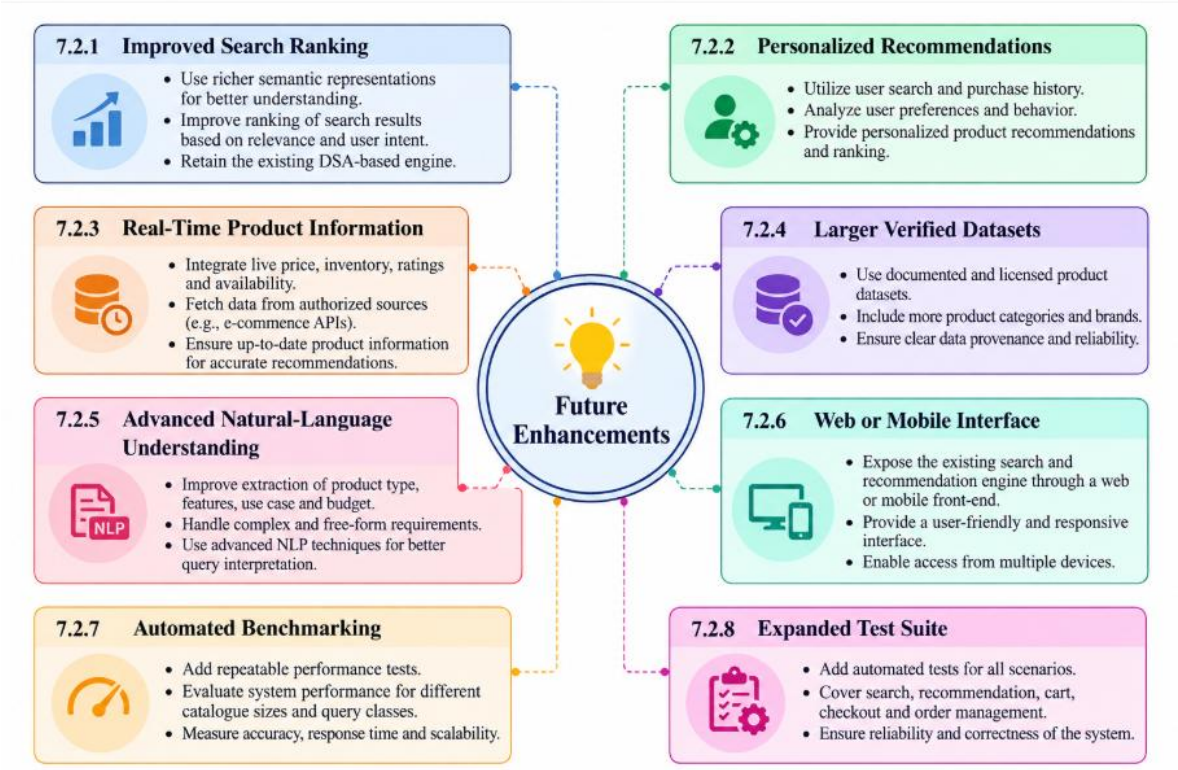
Expose the existing search/recommendation engine through a web or mobile front-end.

7.2.7 Automated Benchmarking

Add repeatable performance tests for different catalogue sizes and query classes.

7.2.8 Expanded Test Suite

Add automated tests for every search, recommendation, cart and checkout scenario.



7.3 SKILLS LEARNED

- Problem Analysis and Algorithm Selection
- String Matching and Pattern Processing
- Dynamic Programming and Edit-Distance Calculation
- Suffix Structures and Similarity Processing
- Graph-Based Matching and Maximum Flow
- Optimization and Approximation
- Randomized Algorithms
- Parallel Processing and Java Concurrency
- Java Swing GUI Development
- CSV Parsing and Data Indexing
- Authentication and File-Based Persistence
- Testing and Debugging
- Technical Documentation
- Git/GitHub-oriented Project Organization

REFERENCES

[1] D. E. Knuth, J. H. Morris Jr., and V. R. Pratt, “Fast Pattern Matching in Strings,” SIAM Journal on Computing, vol. 6, no. 2, pp. 323–350, 1977.

[2] R. M. Karp and M. O. Rabin, “Efficient Randomized Pattern-Matching Algorithms,” IBM Journal of Research and Development, vol. 31, no. 2, pp. 249–260, 1987.

[3] V. I. Levenshtein, “Binary Codes Capable of Correcting Deletions, Insertions, and Reversals,” Soviet Physics Doklady, vol. 10, pp. 707–710, 1966.

[4] J. E. Hopcroft and R. M. Karp, “An n^(5/2) Algorithm for Maximum Matchings in Bipartite Graphs,” SIAM Journal on Computing, vol. 2, no. 4, pp. 225–231, 1973.

[5] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, MIT Press.

[6] Oracle Java Documentation, Java language, Java Swing, and standard Java libraries.

[7] DATA STRUCTURES AND ALGORITHMS - 3 (25CS2103E), course handout and project requirements.

[8] Product Search and Recommendation System for Shopping project source code, data/products.csv, and test evidence.